GAMEPLAY PROGRAMMER PORTFOLIO

GoHome

게임플레이 프로그래머 포트폴리오

엄태호

↗ GitHub / KimHelpMe

E-Mail / sssg2248@gmail.com

Unreal Engine 5.7 / C++

5인 팀 프로젝트 · 개인 구현 및 개선 사례

GoHome 프로젝트 개요

심해 시설에서 물자를 회수하고 잠수정으로 귀환하는 2–4인 협동 생존 게임

개발 기간

2026.08.07 – 2026.10.16

개발 인원

5명

엔진 / 언어

Unreal Engine 5.7 / C++

플랫폼

PC Windows / Steam

본인 역할

게임플레이 프로그래머
레벨 1–3 전체 제작

↗ 프로젝트 저장소

github.com/KimHelpMe/Project_GoHome



개인 기능 종합 시연 영상 게임 플레이 영상

개인 기여와 핵심 과제

게임플레이 시스템 구현 · 레벨 1–3 전체 제작

01

협동 운반

두 플레이어의 입력을
하나의 운반 동작으로 결합

주요 작업·적용 기술

이동·회전 역할 분리
손잡이 중심 추적
서버 이동과 상태 복제

06–10쪽

02

배전반 퍼즐

요청자·상태 검증과
정답·오답 결과 분기

주요 작업·적용 기술

Server RPC 검증
퍼즐 상태 관리
1회용 차단기와 보상 연동

11–13쪽

03

아이템·인벤토리

획득부터 납품까지
아이템 상태를 관리

주요 작업·적용 기술

DataAsset·Actor·Component
책임 분리
중복 등록 방지
슬롯 복제 이벤트와 납품 연동

14–15쪽

04

레벨 1–3 제작

레벨 1·2·3의
전체 맵 제작

주요 작업·적용 기술

Unreal 레벨 에디터
공간 구성과 에셋 배치
레벨 콜리전 수정

18쪽

게임 플레이 흐름

물자 가치·생존 상태에 따른 탐사와 귀환 반복

준비
보급품·목적지 선택

→

시설 탐사
상호작용·위험 대응

→

물자 회수
아이템 확보·협동 운반

→

귀환·정산
납품·가치 반영

개인 담당 시스템의 연동 지점

상호작용 대상 탐지

조준 대상 선정과
서버 상호작용 요청

UInteractionComponent / IInteractable

획득과 운반

아이템 슬롯 관리와
2인 운반 입력 처리

UInventoryComponent / CoopCarry

납품 연동

납품 지점 상호작용과
아이템 가치 전달

DeliveryPoint / GameState 연동

상호작용 클래스 구조

공통 인터페이스 호출과 액터별 상호작용 처리



협동 운반의 초기 문제

두 사람이 함께 들어도 이동이 자주 끊기는 운반 방식

초기 구현: 두 사람의 이동 입력을 평균

초기 Tick() 발췌 · 입력 평균 / 정체 시 해제

```
const FVector InputA = CharacterA->GetLastCarryInputWorld();
const FVector InputB = CharacterB->GetLastCarryInputWorld();
const FVector CombinedInput = (InputA + InputB) * 0.5f;

// ... stuck timer ...
if (TimeStuck >= StuckDropDuration)
{
    ReleaseCarriers();
}
```

상반된 입력에 따른 이동 상쇄
상대 입력에 따른 이동 의도 희석

플레이 중 드러난 불편

방향 불일치로 이동 정체
거리 초과·정체 판정에 따른 운반 해제
물체를 다시 잡는 과정에서 운반 흐름 단절

설계 판단

실패 원인 파악과 대응의 어려움
협동의 재미보다 큰 조작 스트레스



의도치 않은 해제의 원인과 수정

이동 오류를 먼저 수정한 뒤, 운반 조작 자체를 재설계

거리 초과를 유발한 속도 불일치

개인 스프린트로 운반자 간 수영 속도 차이 발생
동일한 입력에도 운반자 간 거리 증가
거리 제한 초과에 따른 의도치 않은 운반 해제

운반 중 개인 스프린트 차단
운반 시작 시 서버·소유 클라이언트의
기존 스프린트 상태 동시 해제

SetCoopCarryObject() · 운반 시작 분기

```
if (NewCarryObject)
{
    ApplySprintState(false);
    if (!IsLocallyControlled())
    {
        Client_ForceStopSprint();
    }
}
```

증상	확인한 원인	수정
클라이언트 입력 미전달	운반 상태의 복제 등록 누락	DOREPLIFETIME 등록
이동 무시·중복 적용	서버 직접 이동과 개인 입력 중복	로컬 폰에서 합산 입력만 적용
다시 잡으면 자동 이동	이전 운반 세션의 입력값 잔존	잡기·놓기 시 입력 초기화

이동·회전 역할과 공용 시점

이동·회전 역할 분담과 공용 3인칭 시점

이동·회전 역할 분리

두 운반자의 조작 역할 분담
이동 담당: 전후·상하 이동
회전 담당: 방향·기울기·공유 부스트
역할별 입력으로 운반 동작 완성

같은 화면에서 방향을 판단

공용 3인칭 카메라로 진행 방향 공유
두 플레이어의 시점·진행 방향 통일
방향 전달·판단 부담 완화
손잡이 양쪽에 마주보는 캐릭터 배치

운반 시연 영상



두 클라이언트의 공용 화면과 역할 HUD

Rotator · 회전 담당

A / D: 좌우 조향
W / S: 상하 기울기
Shift: 공유 부스트

Mover · 이동 담당

W / S: 전진·후진
Space / Ctrl: 상하 이동

운반 시작 시 역할 배정 · 해제까지 유지

손잡이 추적과 조향 안정화

공유 이동과 손잡이 추적으로 운반자·물체의 동작 일치

CoopCarryObjectBase::Tick() · 손잡이 추적 입력

```
const FVector CorrectionA = ((HandleForA->GetComponentLocation()
    - LocationA) / HandleFollowRange).GetClampedToMaxSize(1.f);
const FVector CorrectionB = ((HandleForB->GetComponentLocation()
    - LocationB) / HandleFollowRange).GetClampedToMaxSize(1.f);

CharacterA->SetCombinedCarryInput(SharedMovement + CorrectionA);
CharacterB->SetCombinedCarryInput(SharedMovement + CorrectionB);
```

이동 입력 기반 손잡이 추적

공유 이동에 손잡이 방향 보정 입력 추가
로컬 폰의 AddMovementInput 적용
기존 클라이언트 예측 이동 유지



역할 입력에 따라 이동하는 실제 운반 장면

손잡이 추적 오차에 따른 조향 감속·정지

캐릭터 지연 시 물체 회전 속도 제한
운반자와 물체의 위치 이탈 억제

회전: 새 겹침 검사 / 평행 이동: Sweep

변경 후 동작과 유지한 위험

역할 분담·공용 시점 적용과 피격 시 운반 해제 유지

항목	초기	변경 후
이동 입력	두 입력을 평균	이동·회전 역할 분리
시점	각자 1인칭	공용 3인칭 시점
정체	상쇄 상태가 지속되면 해제	정체 시간에 따른 해제 제거
개인 가속	속도 차이로 거리 이탈	개인 스프린트 차단·공유 부스트
피격	HP 감소 시 함께 해제	동일한 해제 규칙 유지

거리 제한 초과 시 운반 해제 유지

설계 개선의 핵심

오류 수정 후에도 남은 평균 입력의 조작 불편
역할 분담·공용 시점으로 조작 의도 명확화
피격 시 함께 놓치는 위험 유지



몬스터 피격 시 운반 해제

HandleHPChanged() · HP 감소 시 해제

```
if (LastKnownHP >= 0.f &&
    CurrentHP < LastKnownHP)
{
    HandleForcedCarryRelease();
}
```

몬스터 피격 등 HP 감소에 반응

배전반 퍼즐의 상태와 보상

입력 결과와 차단기 조작에 따른 상태·보상 분기

키패드 입력

서버에서 요청자·상태 검증

오답

Locked

Risky 등급 입구 개방

위험 게이지 유지

구역 페널티 유지

정답

Resolved

퍼즐 해결 상태로 전환

해결 후 차단기

위험 초기화 · High 등급 입구

오답·보상 입구 장면



오답 입력 / Failed 표시



보상 입구

오답: Locked 전환 · Risky 입구 개방

정답: Resolved 전환 후 차단기 조작

차단기: 위험 초기화 · High 입구 개방

퍼즐 시연 영상

배전반 입력 요청의 서버 검증

현재 조작자·퍼즐 상태 검증 후 정답 판정

ServerSubmitPassword_Implementation() 소스 ↗

```
220 if (InInstigator != FocusingPawn) return;
221 if (SwitchboardState != ESwitchboardState::PuzzleActive)
      return;
223 if (EnterDigits == PasswordDigits) {
225   HandlePuzzleSucceeded();
226 } else {
229   HandlePuzzleFailed();
230 }
```

ElectricSwitchboardActor.cpp · 220–230행 발췌

검증을 통과한 요청만 정답·오답 처리



현재 조작 중인 키패드와 비밀번호 입력 UI

입력 검증 기준

요청자 현재 조작자 일치

상태 비밀번호 입력 가능 상태

배전반 해결 결과와 1회용 차단기

Resolved 상태와 1회용 차단기·보상 입구 연동

TryResolveViaBreaker() 소스 ↗

```
262 if (!CanFlipBreaker()) { return false; }
267 bBreakerFlipped = true;
268 DangerGauge = 0.f;
271 for (ARewardEntranceActor* Entrance : LinkedEntrances) {
273   if (Entrance) {
275     Entrance->ServerOpen(ERewardGrade::High);
276   }
277 }
278 return true;
```

ElectricSwitchboardActor.cpp · 262–268 / 271–278행 발췌

차단기 1회 조작으로 위험 초기화·High 입구 개방



정답 입력 직후 / Succeeded 표시



조작 전·실패: 차단기 잠김

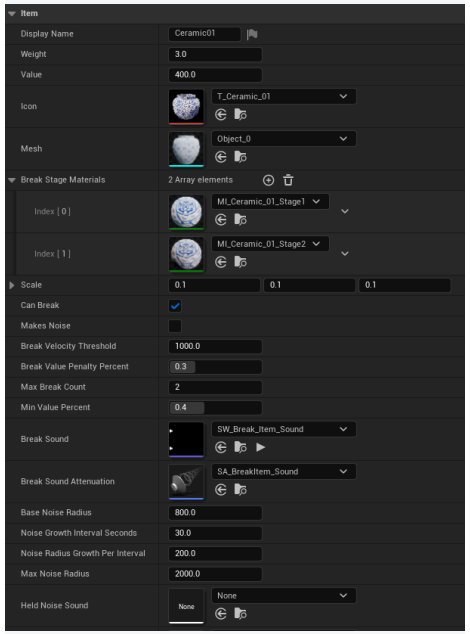


성공: 위험 초기화·입구 개방

아이템 데이터와 인벤토리 구조

설정 데이터·월드 상태·슬롯 관리·납품 연동의 책임 분리

UItemDataAsset	→	AItemActorBase	→	UInventoryComponent	→	GameState 연동
표시·무게·가치 설정		획득·놓기·파손·소음		슬롯·선택·사용 관리		납품 가치 반영



실제 아이템 DataAsset 설정

설정과 상태의 분리

DisplayName / Icon

표시 이름과 아이콘

Weight / Value

무게와 기준 가치

bCanBreak / bMakesNoise

파손·소음 유형

월드 상태와 슬롯

Actor·Component에서 관리



아이템 정보를 표시한 실제 게임 화면

중복 등록 방지와 슬롯 변경 전달

아이템·인벤토리 / 등록 조건과 복제 이후의 변경 이벤트

TryAddItem() · 중복과 빈 슬롯 검사 소스 ↗

```
64 for (const FInventorySlot& Slot : Slots) {
66   if (Slot.Item == Item) return false;
67 }
69 const int32 EmptyIndex = FindEmptySlotIndex();
70 if (EmptyIndex == INDEX_NONE) {
72   return false;
73 }
```

InventoryComponent.cpp · 64–73행 발췌

OnRep_Slots() · 클라이언트 변경 알림 소스 ↗

```
54 OnInventoryChanged.Broadcast();
```

InventoryComponent.cpp · 54행 발췌

문제와 설계 이유

동일 아이템의 슬롯 중복 등록 차단
빈 슬롯이 없으면 등록 중단
슬롯 복제 후 변경 이벤트를 HUD에 전달

획득 후 슬롯



획득한 아이템이 있는 슬롯

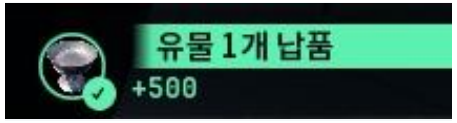


획득 알림 · 팀 HUD 연동

납품 후 슬롯



납품 이후 비워진 슬롯



납품 알림 · 팀 HUD 연동

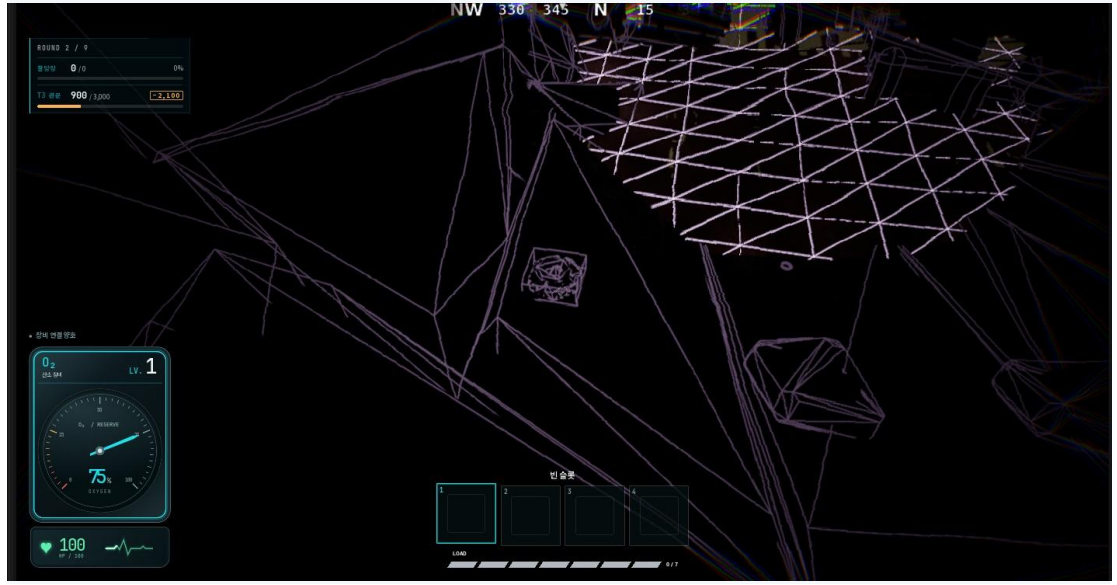
인벤토리 시연 영상

레벨 콜리전에 겹친 아이템 조준 수정

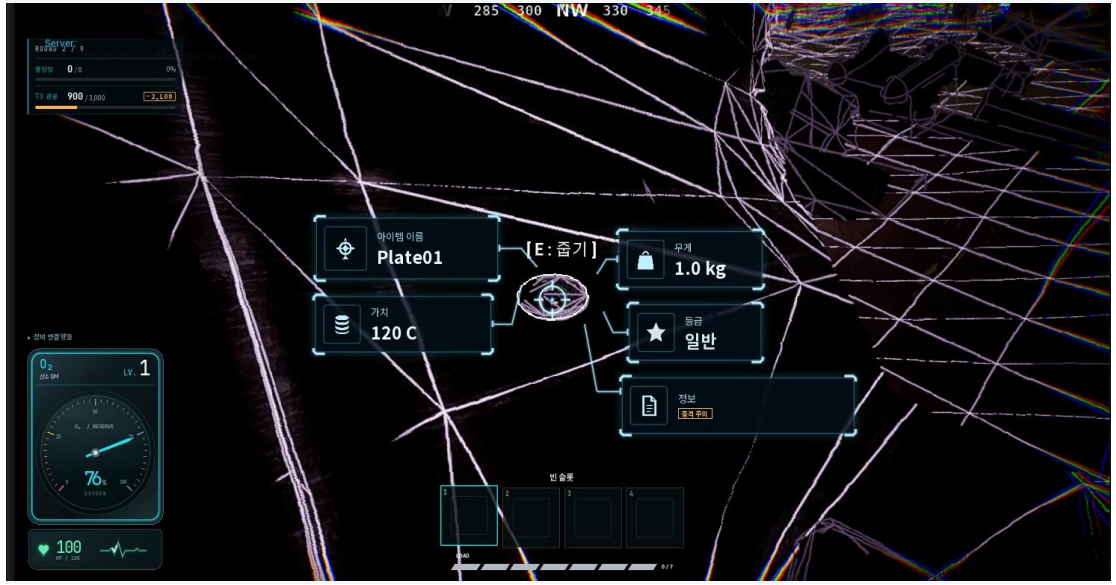
개선 사례

레벨 콜리전이 아이템보다 먼저 시선 검사를 차단
상호작용 대상 탐지 실패

수정 전 / 대상 인식 실패



수정 후 / 겹친 아이템 선택



조준 보정의 탐색 범위 제한

기존 조준 우선 · 탐지 실패 시 막힌 지점 주변 추가 탐색

선 트레이스
정확히 조준한 대상 우선

→

구체 스윕
살짝 빗나간 조준 보정

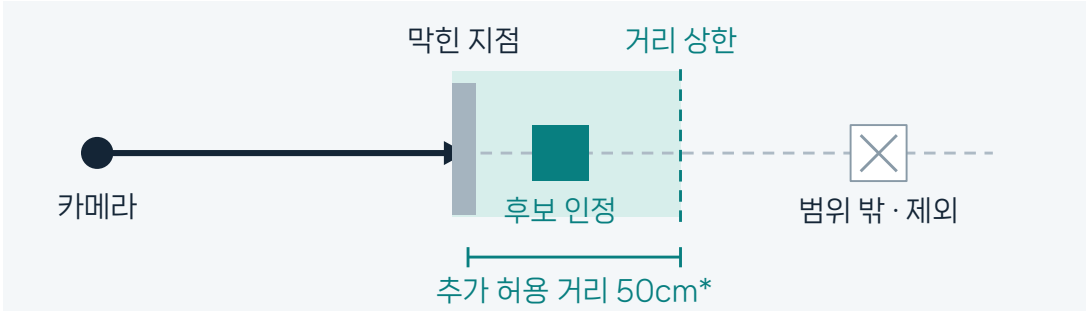
→

막힌 지점 근처 탐색
추가 허용 거리 안에서 후보 선택

FindItemBehindBlocker() · 핵심 조건 소스 ↗

```
246 const float MaxDist = FMath::Min(TraceDistance,
      BlockDistance + OverlapAimTolerance);
250 float BestDist = MaxDist;
  … // … 후보 검사 준비·반복문 생략
262 if (Item->MeshComponent->SweepComponent(
        ItemHit, Start, End, FQuat::Identity, Shape, false)
263     && ItemHit.Distance <= BestDist) {
265   BestDist = ItemHit.Distance;
266   Best = Item;
267 }
```

InteractionComponent.cpp · 246·250·262–267행 발췌



기존 조작감 유지

선 트레이스·구체 스윕 우선
기존 검사에서 탐지 실패 시
추가 아이템 탐색

과도한 보정 억제

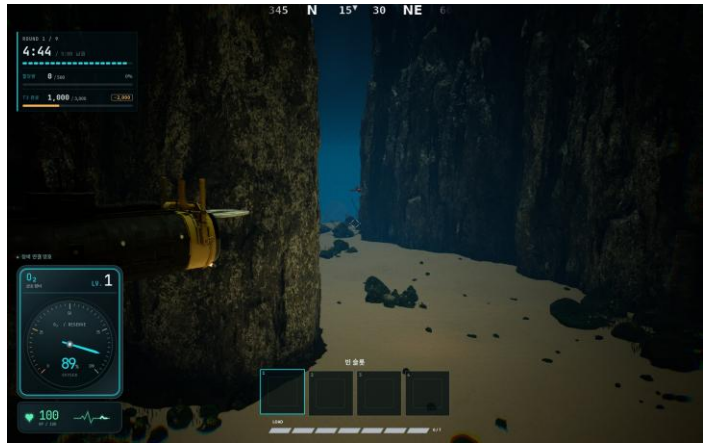
막힌 거리 + 허용치와
전체 사거리 중 작은 값으로
탐색 거리 상한 제한

남은 한계 허용치 확대 시 얇은 벽 너머 아이템 선택 가능

레벨 1–3 전체 제작

레벨 1·2·3 전체 맵 제작

LEVEL 01



LEVEL 02



LEVEL 03



레벨 1–3 전체 제작

레벨 1·2·3의 전체 맵 제작
공간 구성과 에셋 배치

레벨 콜리전 수정

레벨 2 대형 선박의
콜리전 통과 문제 수정

파손·오디오 개선

파괴 벽 타격음·아이템 파손음 추가
텔레포트 사운드 감쇠 범위 수정

↗ 레벨 스크린샷

GAMEPLAY PROGRAMMER

연락처

엄태호

↗ GitHub / KimHelpMe

↗ 프로젝트 저장소 / github.com/KimHelpMe/Project_GoHome

E-Mail / sssg2248@gmail.com